Übung 6: Prioritätswarteschlange, Heap

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie können Heaps und Prioritätswarteschlangen für unterschiedliche Problemstellungen einsetzen, Heap-Operationen durchführen und deren Laufzeiten analysieren. Außerdem können Sie geeignete Datenstrukturen für konkrete Anwendungen auswählen und begründen.

1. Ternäre Heaps

   Betrachten Sie die folgende Zahlenfolge:

   $$5, 20, 10, 6, 7, 3, 1, 2$$

   (a) Fügen Sie die folgenden Zahlen nacheinander in einen ternären Min-Heap ein. Ein Knoten kann dabei bis zu drei Kinder haben. Zeichnen Sie die Baumdarstellung Ihres fertigen ternären Heaps.

   (b) Geben Sie die Array-Darstellung des resultierenden Heaps an. Verwenden Sie dabei die Indizierung ab Index 0.

   (c) Geben Sie für $i > 0$ eine Formel an, mit der sich der Index des Elternknotens eines Knotens mit Index i bestimmen lässt.

   (d) Geben Sie eine Formel an, mit der sich der Index des j-ten Kindes eines Knotens mit Index i bestimmen lässt. Nummerieren Sie die Kinder dabei mit $j = 0, 1, 2$.

2. Heaps

   (a) Fügen Sie die folgenden Schlüssel nacheinander in einen Min-Heap ein:

   $$[10, 7, 15, 17, 12, 20, 6, 32]$$

   (b) Fügen Sie nun die gleichen Schlüssel in einen Max-Heap ein.

   (c) Erzeugen Sie nun aus denselben Schlüsseln mit dem buildHeap-Algorithmus von Floyd einen Min-Heap.

   (d) Fügen Sie $[1, 0, 1, 1, 0]$ in einen Min-Heap ein.

   (e) Führen Sie die Operation removeMin auf dem folgenden Min-Heap aus: $[2, 5, 7, 8, 10, 9]$. Geben Sie das resultierende Array an.

3. Sortieren mit Heaps

   (a) Muss das Array, das einen Heap repräsentiert, sortiert sein? Begründen Sie Ihre Antwort.

   (b) Angenommen, ein Array ist aufsteigend sortiert. Muss es die Array-Darstellung eines gültigen Min-Heaps sein? Begründen Sie Ihre Antwort.

   (c) Gegeben sei die Array-Darstellung eines Min-Heaps. Ist das umgekehrte Array stets die Array-Darstellung eines gültigen Max-Heaps? Begründen Sie Ihre Antwort anhand eines Beispiels.

   (d) Wie können Sie die Array-Darstellung eines Min-Heaps möglichst effizient in einen Max-Heap umwandeln? Geben Sie einen geeigneten Algorithmus an und bestimmen Sie dessen Laufzeit in der O-Notation.

4. Ist das ein gültiger Heap?

   Gegeben seien die folgenden Arrays:

$$A = [1, 4, 3, 7, 6, 8, 5]$$
$$B = [1, 3, 2, 7, 6, 5, 8]$$
$$C = [8, 7, 6, 5, 4, 3, 2]$$
$$D = [8, 7, 6, 5, 9, 3, 2]$$

   (a) Entscheiden Sie für jedes Array, ob es die Array-Darstellung eines gültigen Min-Heaps bzw. Max-Heaps ist.

   (b) Begründen Sie Ihre Entscheidungen anhand der Heap-Eigenschaft.

   (c) Für jedes Array, das kein gültiger Min- bzw. Max-Heap ist: Geben Sie einen Knoten an, an dem die Heap-Eigenschaft verletzt ist.

   (d) Entwerfen Sie einen Algorithmus, der für ein gegebenes Array überprüft, ob es einen gültigen Min-Heap darstellt. Geben Sie den Algorithmus in Pseudocode an.

   (e) Bestimmen Sie die Laufzeit Ihres Algorithmus in der O-Notation.

5. Sie arbeiten an einer Anwendung zur Verwaltung von Kundenaufträgen in einem Online-Shop. Jeder Kundenauftrag hat eine eindeutige Auftragsnummer, einen Kundenname, einen Bestellwert und eine Priorität. Die Anwendung soll in der Lage sein, die folgenden Operationen effizient auszuführen:

   - Einfügen eines neuen Kundenauftrags mit allen seinen Daten.
   - Suchen eines Kundenauftrags anhand der Auftragsnummer.
   - Aktualisieren des Bestellwerts oder der Priorität eines Kundenauftrags.
   - Entfernen des Kundenauftrags mit der höchsten Priorität aus der Warteschlange.

   (a) Wählen Sie einen geeigneten abstrakten Datentyp und eine oder mehrere geeignete Datenstrukturen, um diese Operationen effizient zu unterstützen. Begründen Sie Ihre Wahl.

   (b) Definieren Sie einen geeigneten Schlüssel für den abstrakten Datentyp und erläutern Sie, wie Sie die Eindeutigkeit der Schlüssel sicherstellen.

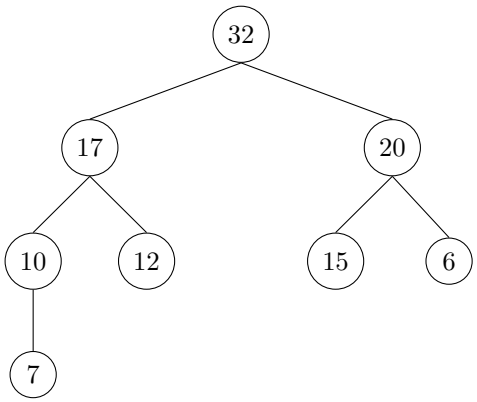
   (c) Geben Sie für jede der vier Operationen die erwartete Laufzeit in der O-Notation an.

6. Sie arbeiten für eine sehr große Notaufnahme eines Krankenhauses. Normalerweise werden in einer Notaufnahme Patienten in der Reihenfolge ihres Eintreffens und des Schweregrads ihrer Verletzung behandelt. Nehmen Sie an, dass jede Patient*in als Schlüssel-Wert-Paar abgespeichert wird. Wie könnten Sie den **Schlüssel** modellieren und was könnte als **Wert** im Schlüssel-Wert-Paar abgespeichert werden? Wählen Sie dann einen abstrakten Datentyp (ADT) und eine Datenstruktur, um die Patienten im Computer der Notaufnahme in der richtigen Reihenfolge abspeichern zu können. Bitte **begründen** Sie kurz Ihre Wahl des Schlüssels. Die Wahl des ADTs und der Datenstruktur müssen Sie nicht begründen.

7. Erzeugen Sie aus der folgenden Zahlenfolge mithilfe des buildHeap-Algorithmus von Floyd einen Max-Heap. Markieren Sie Ihre finale Lösung.

$$[10, 12, 20, 6, 7, 15, 17, 32]$$

8. Entfernen Sie das größte Element aus dem folgenden Max-Heap und zeichnen Sie den resultierenden Max-Heap. Markieren Sie Ihre finale Lösung, falls Sie mehrere Heaps als Zwischenschritte zeichnen.



9. Ein Unternehmen möchte sicherstellen, dass seine Server regelmäßig gewartet werden. Es hat folgende Funktion `find_oldest_server` implementiert, die in einer for-Schleife alle Server durchgeht und den Server zurückgibt, der das älteste letzte Wartungsdatum hat.

   Die Funktion soll künftig wiederholt aufgerufen werden. Die Server sollen deshalb in einer geeigneten Datenstruktur gespeichert werden.

   Optimieren Sie die Funktion `find_oldest_server` durch die Speicherung der Server in einer geeigneten Datenstruktur und beschreiben Sie wie die neue Implementierung der Funktion `find_oldest_server` aussehen würde. Eine Beschreibung als Satz oder Pseudocode genügt. Python-Code ist alternativ auch möglich. Bestimmen Sie ebenfalls die Laufzeit der aktuellen Funktion `find_oldest_server` und die der neuen Implementierung in der O-Notation.

```
from datetime import datetime

class Server:
  def __init__(self, name, last_maintenance):
    self.name = name
    # Das Datum der letzten Wartung wird in dieser Variablen gespeichert
    self.last_maintenance = last_maintenance

  # ermöglicht Ausdrücke wie server1 < server2, wobei server1 und server2 Server Objekte sind
  def __lt__(self, other):
    return self.last_maintenance < other.last_maintenance

  def find_oldest_server(servers):
    oldest_server = servers[0]
    for server in servers:
      if server.last_maintenance < oldest_server.last_maintenance:
        oldest_server = server
    return oldest_server

# Beispiel für die Nutzung: 3 Server mit zugehörigen Datumsangaben der letzten Wartung
servers = [
  Server("Server1", datetime(2023, 5, 1)),
  Server("Server2", datetime(2023, 6, 10)),
  Server("Server3", datetime(2024, 3, 5))
  ]

oldest = find_oldest_server(servers)
print(f"Der Server mit dem ältesten Wartungsdatum ist: {oldest.name}")
```